

C-DAWN: Keeping a Battery-Limited UAV Swarm Connected While It Surveys a Himalayan Disaster Valley

Team C-DAWN | [Member names] | [Institution, City]

Source code: github.com/paramvansh01/drone_swarm | Simulation track | 26 September 2026

ABSTRACT. When an earthquake or a landslide cuts the road and the towers in a Himalayan valley, the first hours depend on two things: getting eyes on the damaged sites, and getting what those eyes see back to the people running the response. We built C-DAWN, a simulation-first autonomy stack for a fleet of battery-limited quadrotors that fly from a ground control station (GCS) outside the affected area. The swarm works out from the terrain how many relays it needs, uses an E(3)-equivariant graph network and a differentiable radio model to decide where they hover, hands a relay station to a fresh aircraft *before* the tired one leaves to swap its battery, and drops lower-priority work when a new emergency is reported. It is never told about a failure. It notices one from heartbeats, acknowledgements, noise readings and battery current, the same evidence a fielded swarm would have, and a causal model with small altitude experiments tells terrain shadow apart from interference. Across four scenarios, two of them on real terrain at Kedarnath and Uttarkashi, with 4 to 11 hidden disturbances each, C-DAWN delivered 97.3% of survey tasks, 96.7% of packets and 95.3% connectivity, against 87.8%, 91.6% and 88.6% for a fixed-role baseline. Neither had a collision, a geofence violation or a flat battery.

Keywords: UAV swarm, BVLOS, multi-hop relay, equivariant graph neural network, differentiable RF, structural causal model, energy-aware planning, delay-tolerant networking

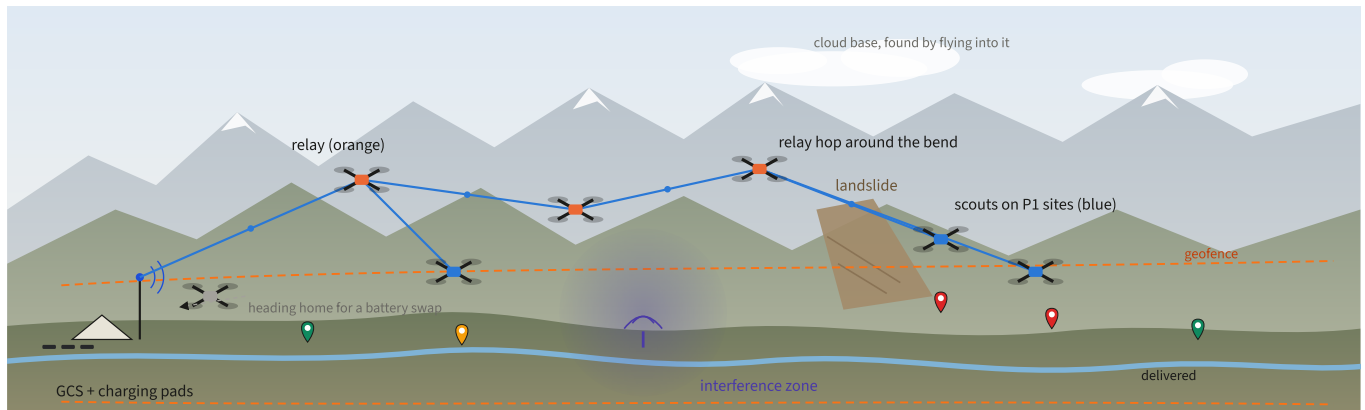


Fig. 1. What C-DAWN is doing. The GCS sits at the valley mouth with its charging pads. Relays (orange) hold a chain that follows the valley round its bend; scouts (blue) survey the reported sites, coloured by priority, which turn green once their data has reached the GCS. One aircraft is heading back for a battery swap while its station stays covered. The interference zone, the cloud base and the geofence are all things the swarm has to find out about or respect on its own.

1 INTRODUCTION

Mountain valleys are hard places to keep a radio link alive. A 900 MHz link at half a watt reaches tens of kilometres in open air, but in the Mandakini or Bhagirathi valleys one spur of rock can take 40 dB out of it. Add a battery that lasts a quarter of an hour, a valley wind that makes the trip home twice as slow as the trip out, and the ordinary ways a deployment goes wrong (an aircraft goes down, a radio dies, somebody's generator sprays noise across the band), and the UAV-X problem comes into focus. The fleet has to survey every reported site and keep talking to the GCS the whole time, without anyone telling it what has just broken.

Most multi-UAV search systems [11] plan the survey first and check the radio afterwards. We treat communication as part of the mission instead. A site only counts as done when its imagery has reached the GCS, so the swarm is scored on exactly what responders need. Our contributions are:

1. A **terrain-aware relay requirement**: a chain walk over the elevation map decides how many relays the valley needs, instead of a constant.
2. **Learned relay placement**: an E(3)-equivariant graph network trained without labels through a differentiable terrain

and link model, then polished by gradient steps at run time.

3. **Dynamic roles** with 200 to 300 ms failover and a **make-before-break handover**, so a relay leaving to recharge never opens a gap.
4. **Energy- and wind-aware planning** driven by each aircraft's own measured current draw.
5. A **perception-only awareness layer** with causal link diagnosis. The autonomy never reads simulator truth, and a test audits the source to keep it that way.
6. A reproducible evaluation: JSON scenarios, hidden disturbances, standard logs and a fixed-role baseline.

2 PROBLEM AND SYSTEM MODEL

Area and GCS. The affected area is a valley corridor about 4.2 km long inside a 5.1×5.1 km terrain tile. The GCS is a fixed node at the valley mouth with its antenna on a 10 m mast and a row of charging pads. It cannot fail, every packet is going there, and every sortie starts and ends on one of its pads.

Fleet. Five or six identical 1.9 kg quadrotors with a 900 MHz mesh radio (27 dBm, 3 dBi). Hover endurance is about 18 minutes and cruise about 15; a battery swap takes up to two min-

Table 1. How each UAV-X requirement is met and how we measure it.

The swarm must	What C-DAWN does	Where	Measured as
Survey all assigned locations	Assign a task only if the aircraft can reach it, survey it and get home inside its battery and the clock; greedy on priority weight per second of travel (Sec. 5.2).	sim/guidance.py sim/energy.py	completion rate and time, priority-weighted score
Keep end-to-end comms with the GCS	GCS as a fixed graph node; chain sized from terrain; GNN placement; causal-aware routing; store-and-forward traffic.	mesh/roles.py gnn/ mesh/traffic.py	PDR, latency, availability, downtime
Assign relays dynamically	Relay count re-planned every second; launch from pads first, else promote the scout doing the least valuable work.	mesh/roles.py	relay reallocations, reconfiguration efficiency
Reconfigure on degradation, failure or recharge	Heartbeat failover; interference localised and routed round; make-before-break handover; lost-link failsafe.	mesh/election.py mesh/interference.py	recovery time, performance after faults
Prioritise new high-priority regions	A new P1 task pre-empts the best-placed scout on lower-priority work; standby scouts; discretionary returns turned round.	sim/guidance.py	emergency response time, weighted score
Finish safely, on time	Energy-aware return; home before the deadline; geofence projection on every setpoint; predictive separation; spaced take-offs.	sim/deconfliction.py sim/world.py	collisions, minimum separation, flat batteries, geofence breaches

utes. Roles are not fixed: an aircraft can be a scout, a relay, or on standby.

Tasks. A survey task p has a position, a priority $\pi_p \in \{1, 2, 3\}$ with weight $w_p = 3, 2, 1$, and a release time. Tasks released after launch are the newly emerging ones, invisible to the swarm until reported. A task is surveyed after a 2 s dwell within 30 m and *delivered* when all 12 chunks of its data are at the GCS.

Constraints. A mission clock of 12 or 15 minutes, a polygon geofence with an AGL ceiling, at least 3 m between aircraft, and a battery that never runs out in the air. We read “finish safely within the allotted time” strictly: every aircraft is back on a pad by the deadline.

3 ARCHITECTURE

Everything runs in one Python process on one simulated clock. The live console, a three-laptop demonstration and the headless benchmark all call the same `step()`, so the numbers in Section 7 come from the code that is shown live. Fig. 2 shows the layers. The line we care most about is the dashed one. Below it is the simulator, which knows everything. Above it is the autonomy, which only knows what its sensors and its radio tell it. Measurements are the only thing that crosses upward, and setpoints are the only thing that crosses back down.

4 SIMULATION ENVIRONMENT

Terrain. Each site is a 5.1 km terrain pack with a 5 m elevation grid built from AWS Open Data terrain tiles (SRTM and Copernicus derived) and Sentinel-2 cloudless imagery [9, 10]. We ship four real disaster sites (Kedarnath, Uttarkashi, Joshimath and Chungthang) and a procedural valley for repeatable tests. The mission corridor is found by dynamic programming along the valley floor.

Flight. An underactuated multirotor with attitude lag, slew limits and airspeed drag, flown by a cascaded PID. Wind follows a power-law shear, $w(z) = w_{\text{ref}}(z/z_{\text{ref}})^{0.16}$, with Dryden turbulence and gusts [8].

Radio. Every tick, every pair of aircraft and every aircraft-to-GCS link gets a link budget

$$P_r = P_t + G_t + G_r - L_{\text{fs}}(d) - J(v) - L_x, \quad \text{SNR} = P_r - N, \quad (1)$$

where L_{fs} is Friis free-space loss [2], L_x extra loss (wet antennas in rain) and $J(v)$ the ITU-R P.526 knife-edge loss over the worst ridge on the path [1]. With h the height of the ridge above the ray

and d_1, d_2 its distances to the two ends,

$$J(v) = 6.9 + 20 \log_{10} \left(\sqrt{(v - 0.1)^2 + 1} + v - 0.1 \right), \quad (2)$$

$$v = h \sqrt{2(d_1 + d_2)/(\lambda d_1 d_2)}.$$

The noise floor N is thermal plus any interference received from active sources. Packet delivery is averaged over a burst of 24 packets, each with its own Rician fade ($K = 6$ dB). We learnt early that one fading draw per tick makes a steady link look like it is flapping, which confused both routing and the causal engine.

5 MISSION AUTONOMY

Before the individual pieces, it helps to see one aircraft’s life (Fig. 3). It starts charged on a pad, launches as a scout or a relay, may swap roles in the air, and eventually comes home for one of five reasons. It lands, has its battery swapped and waits until the role manager needs it again. Everything in this section decides when one of these arrows is taken.

5.1 Energy and time, before anything else

Every decision that sends an aircraft away from the GCS has to be affordable. Drain follows the airframe’s power law, scaled by a factor $\hat{k}$ that each aircraft *measures* as the ratio of its battery current to what its thrust should need, over a 15 s window:

$$\dot{b} = \hat{k} [b_0 + (b_1 - b_0)f_T], \quad v_g = \text{clip}(v_a + \hat{\mathbf{w}} \cdot \hat{\mathbf{u}}, 5, 19), \quad (3)$$

$$\hat{k} \leftarrow \hat{k} + \frac{\Delta t}{15} (I_{\text{meas}}/I_{\text{nom}} - \hat{k}), \quad (4)$$

with f_T the thrust fraction, $\hat{\mathbf{w}}$ the swarm’s wind estimate at cruise height and $\hat{\mathbf{u}}$ the direction of travel. A damaged motor raises I_{meas} , so $\hat{k}$ rises and the aircraft plans more conservatively without being told anything. Leg time is $T(a, b) = 1.15 |ab|/v_g + 25$ s. With safety factor $s = 1.2$ and an 8% landing reserve, the return and handover thresholds are

$$B_{\text{rth}} = \max(12, s \dot{b} T(\mathbf{p}, \mathbf{h}) + 8), \quad (5)$$

$$B_{\text{ho}} = B_{\text{rth}} + s \dot{b}_{\text{hov}}(T(\mathbf{g}, \mathbf{q}) + 25 \text{ s}), \quad (6)$$

where $\mathbf{p}$ is the aircraft, $\mathbf{h}$ its pad, $\mathbf{g}$ the GCS and $\mathbf{q}$ a relay’s station. There is no fixed “come home at 22%” rule. A relay four kilometres up the valley with the wind in its face turns for home at about 62%, one next to the GCS at 12% (Fig. 4).

5.2 Who surveys what, and when to change our mind

Allocation is greedy on value rate. At each step we take the (scout, task) pair with the largest value rate

$$V_{s,p} = \frac{w_p}{\text{ETA}_{s,p} + 60 \text{ s}}, \quad (7)$$

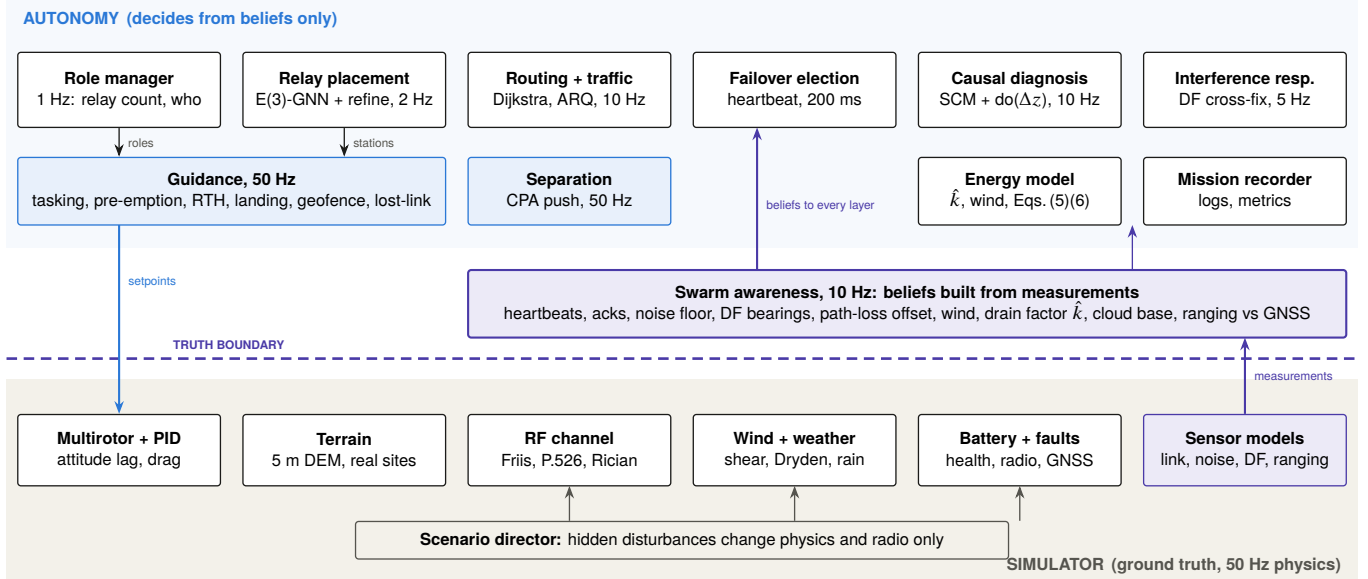


Fig. 2. Layers, rates and the truth boundary. The simulator owns terrain, flight physics, the radio channel, faults and the scenario's hidden disturbances. Only sensor readings cross the purple line. The awareness layer turns them into beliefs and every autonomy layer decides from those. Setpoints are the only thing that goes back down. Rates are per simulated second.

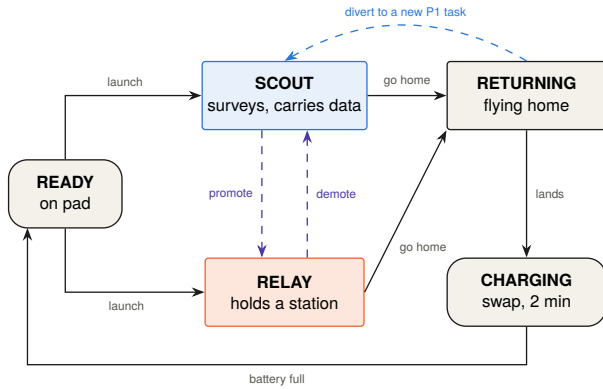


Fig. 3. The life of one aircraft. Dashed purple arrows are role changes in flight, each logged as a reallocation. An aircraft goes home for its energy threshold, the mission clock, a lost link, lack of work, or a completed handover. Failure is not a state anyone announces: an aircraft nobody has heard for 200 ms is treated as gone.

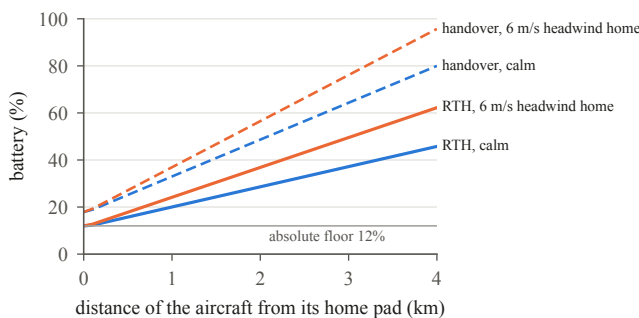


Fig. 4. Thresholds from Eqs. (5) and (6) for a cruising aircraft. A headwind on the way home adds 16 points at 4 km. Dashed lines are handover thresholds, which leave time for a replacement to fly out.

considered only if the task is affordable and the round trip fits in the time left. The 60 s term stops a very close P3 task from always beating a P1 task a little further away. When a P1 task arrives and no idle scout can take it, the best-placed scout on lower-priority work is pre-empted, unless it is already hovering over its target. Idle scouts with a healthy battery loiter at a standby point mid-valley, inside relay cover, so the next emergency is minutes closer. A scout flying home only because it had nothing to do can be

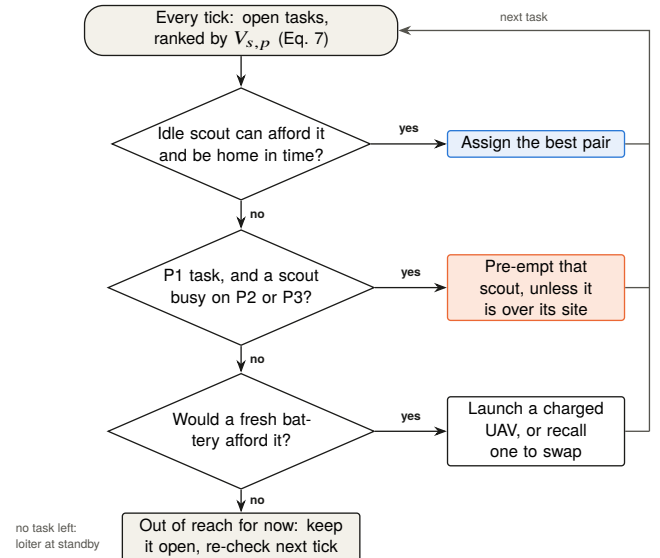


Fig. 5. Task allocation, run every tick. “Afford” means battery for the task, the trip home and the landing reserve; “in time” means the round trip fits before the deadline.

Algorithm 1 Terrain-aware relay chain

Require: GCS antenna $\mathbf{g}$, farthest work point $\mathbf{t}$, elevation map, noise estimate $\hat{N}$, localised sources $\hat{S}$

- 1: sample the valley centreline every 60 m from $\mathbf{g}$ to $\mathbf{t}$ at 160 m AGL
- 2: $\mathbf{c} \leftarrow \mathbf{g}$; hops $\leftarrow []$
- 3: **while** $\text{SNR}(\mathbf{c}, \mathbf{t}) < 15 \text{ dB}$ **do** ▷ Eq. (1) over the map
- 4: $\mathbf{s} \leftarrow$ farthest sample with $\text{SNR}(\mathbf{c}, \mathbf{s}) \geq 15 \text{ dB}$ given $\hat{N}, \hat{S}$
- 5: **if** no such $\mathbf{s}$ **then break**
- 6: append $\mathbf{s}$ to hops; $\mathbf{c} \leftarrow \mathbf{s}$
- 7: **return** |hops| (relay requirement) and hops (GNN anchors)

turned round. Fig. 5 is the whole loop.

5.3 How many relays the terrain asks for

This replaces a guess with a measurement of the mountain. Once a second the role manager collects the points where work is happening: scouts on task, open reachable tasks, aircraft carrying undelivered data and aircraft still flying home. For the farthest of them it walks the chain in Algorithm 1.

The 15 dB target is roughly the 9 dB a 256-byte frame needs

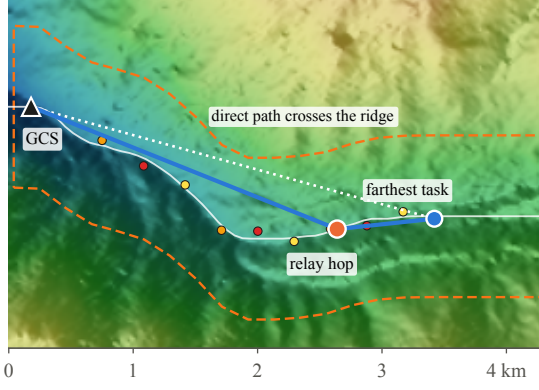


Fig. 6. Algorithm 1 on the real Kedarnath terrain. Blue lines are the planned links; the white dotted line is the direct path, which clips the spur. Dots are survey tasks by priority (red P1, amber P2, yellow P3); the orange dashed line is the geofence.

for 95% delivery, plus fading margin. On Kedarnath the walk gives exactly one relay (Fig. 6): the valley turns south past the 2 km mark and a spur blocks the straight line from the GCS to the far sites.

The requirement then drives assignment. When there are fewer relays than needed, a charged aircraft on a pad launches first, because it arrives with a full battery. Failing that, the scout doing the least valuable work closest to the missing hop is promoted, and at least one scout always keeps surveying. When there are more relays than needed for 25 s, the spare goes back to surveying, or home. Every change is logged with its reason; one undone within 30 s counts as a flap against reconfiguration efficiency.

5.4 Where the relays hover: an equivariant GNN

The role manager decides how many relays and who flies them. Where they hover is decided by a graph network followed by a short gradient polish (Fig. 7). We picked a learned model for one reason: the placement has to be recomputed twice a second as scouts move, and one forward pass costs only 1.4 ms.

Graph and features. Every aircraft that somebody can hear, plus the GCS, is a node; with fewer than ten nodes we use a fully connected graph. Positions $\mathbf{x}_i \in \mathbb{R}^3$ are centred on the swarm centroid and divided by $L = 250$ m, because raw coordinates in kilometres saturate an MLP. Each node carries nine invariant features $\mathbf{h}_i^0$: battery, a one-hot role (scout, relay, GCS), mean and minimum measured link quality, degree, AGL and an interference flag.

Message passing. After a two-layer SiLU embedding to 64 channels, four E(n)-equivariant layers [3] update features and coordinates together:

$$m_{ij} = \phi_m(\mathbf{h}_i, \mathbf{h}_j, \|\mathbf{x}_j - \mathbf{x}_i\|), \quad (8)$$

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} (\mathbf{x}_j - \mathbf{x}_i) \phi_x(m_{ij}), \quad (9)$$

$$\mathbf{h}_i \leftarrow \mathbf{h}_i + \phi_h\left(\mathbf{h}_i, \sum_j m_{ij}\right). \quad (10)$$

ϕ_m and ϕ_h are two-layer MLPs and ϕ_x ends in a tanh, so each layer's step is bounded. Since the only geometric input to ϕ_m is a distance and the coordinate update is a weighted sum of relative vectors, the network satisfies $f(Q\mathbf{x} + \mathbf{t}) = Qf(\mathbf{x}) + \mathbf{t}$ for any rotation or reflection Q and shift $\mathbf{t}$. A model trained on one valley therefore works on a valley running in any direction; unit tests check this numerically, to 1 cm for the full network. We used mean rather than sum aggregation in (9) so the step size does not grow with the number of neighbours.

Table 2. Relay placement network: settings and held-out results (48 layouts, six unseen terrains).

Setting	Value
Layers / hidden width / parameters	4 / 64 / 123,397
Epochs $\times$ layouts, optimiser	220 $\times$ 24, Adam, $8 \cdot 10^{-4}$, cosine
Train / test terrain	different seeds, no overlap
Link softness s (train / flight)	20 dB / 3 dB
Refinement in flight	20 Adam steps, step 6 m
Placement	Connectivity, time
Naive relay line	0.48
GNN forward pass only	0.58, 1.4 ms
GNN + refinement (deployed)	0.66 , 1.4 + 21 ms

Output. A per-node gate $g_i = \sigma(\text{MLP}(\mathbf{h}_i))$ scales the proposed move, $\Delta_i = g_i L(\tilde{\mathbf{x}}_i^{(4)} - \tilde{\mathbf{x}}_i^{(0)})$, which is clipped to 220 m and applied to relays only. Scouts, the GCS and a relay that is being relieved stay fixed. The network has 123,397 parameters.

A differentiable radio. To score a layout without labels we rebuilt Eqs. (1) and (2) in PyTorch so that every step has a gradient. Terrain along each link is read with bilinear `grid_sample`. The worst ridge is a hard max, which would send gradient to one sample only, so we use a log-sum-exp smooth max. ITU-R P.526 is only defined for $v > -0.78$, and a hard cutoff would put a kink exactly where relays spend their time (at the edge of Fresnel clearance), so it is gated by a sigmoid. Link delivery becomes $P_{ij} = \sigma((\text{SNR}_{ij} - \gamma)/s)$ with $s = 3$ dB in flight. End-to-end reliability to scout k is a soft Bellman recursion over at most four hops:

$$r_j^{(t+1)} = \sum_i \text{softmax}_i(\tau c_{ij}) c_{ij}, \quad c_{ij} = r_i^{(t)} P_{ij}, \quad (11)$$

with $\tau = 12$ and $r^{(0)}$ equal to 1 at the GCS; the candidate set includes staying put. As $\tau \rightarrow \infty$ this is the most reliable path. The objective penalises the worst-served scout as well as the average:

$$\mathcal{L} = -(\bar{r} - \frac{1}{2}(\bar{r} - r_{\min})) + \lambda_s S + \lambda_g G + \lambda_e E, \quad (12)$$

where S is squared intrusion below 25 m separation, G squared excursion outside 40 to 480 m AGL and E the squared move, with weights 0.02, 0.015 and 2×10^{-5} in the units used.

Training. The network is trained self-supervised with Adam [12] on randomised layouts: GCS, scouts and a deliberately naive relay line (evenly spaced, low, often in shadow) on a procedural valley. There is no teacher. With the deployed $s = 3$ dB, a link 40 dB below threshold has zero gradient, so “this scout is unreachable” says nothing. We therefore train with $s = 20$ dB and evaluate at 3 dB. Table 2 lists the settings and the held-out results. The refinement does most of the work. The network’s real contribution is a good starting point, which makes 20 refinement steps enough.

5.5 Handing over a relay station

Batteries are why relays leave, and a naive swarm pulls the relay home and then notices the hole. We do it the other way round (Fig. 8). When a relay’s battery reaches B_{ho} from Eq. (6), the role manager launches a charged aircraft, or promotes a scout, and sends it to the relay’s exact station. The outgoing relay is removed from the placement plan and holds where it is. It leaves only once the replacement is holding station. If its own return threshold arrives first, it comes home anyway: safety beats coverage.

5.6 Failover and routing

Failover. An aircraft is “heard” when anyone, another aircraft or the GCS, measures a usable link to it. A relay nobody has heard

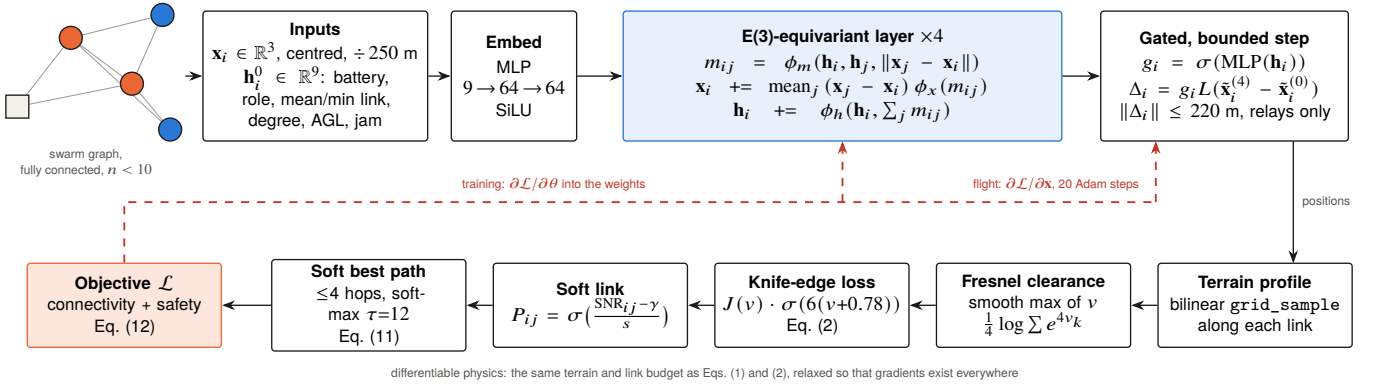


Fig. 7. Relay placement. Top: the forward pass, from swarm graph to a bounded move for each relay. Bottom: the differentiable link model that scores a layout. Red: gradients. During training they update the network weights; in flight the same gradients move the relay positions for 20 Adam steps (about 21 ms). No labelled data is used anywhere.

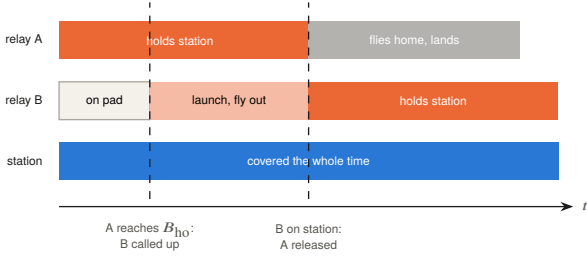


Fig. 8. Make-before-break. The overlap between the dashed lines is the price of never opening a gap; Eq. (6) sizes it so that A can afford to wait and still get home.

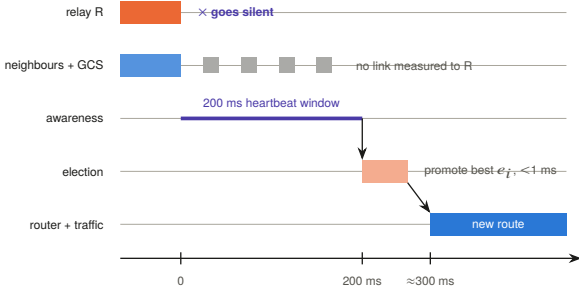


Fig. 9. Failover as it happens. Nobody announces that R has failed. The swarm notices that nobody can hear it; after 200 ms the election fills the slot and the router moves traffic. The election takes microseconds; the waiting is what costs time.

for 200 ms is treated as failed, and the scout with the best score

$$e_i = \frac{b_i}{100} \bar{q}_i \rho_i \quad (13)$$

is promoted into its slot, where b_i is battery, $\bar{q}_i$ mean measured link quality and ρ_i a role weight. The router recomputes at once. Detection dominates the latency (Fig. 9): in our logged runs the whole thing, from silence to traffic on the new route, took 200 to 300 ms. Nothing that still works is demoted. Our first election re-ranked the whole swarm on every failure, and we watched it push a healthy relay out of its role.

Routing. Routes are shortest paths [14] over the flying ad-hoc network [4], with the GCS as a node, on a cost that mixes current quality with how stable the causal model expects a link to be:

$$c_{ij} = (1 - \alpha)(-\log q_{ij}) + \alpha(-\log s_{ij}), \quad \alpha = 0.3, \quad (14)$$

where s_{ij} is an exponential average (weight 0.3) of $1 - \hat{L}_{ij}$ from the SCM in Sec. 6, halved while the link is anomalous. Taking logs makes the path cost the negative log of end-to-end reliability, so Dijkstra finds the most reliable route rather than the shortest.

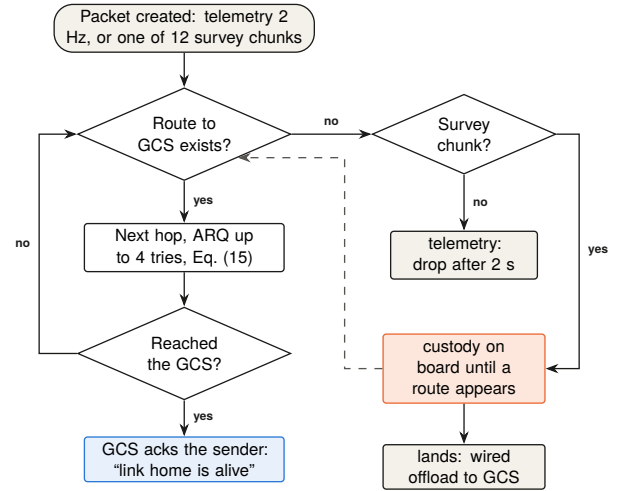


Fig. 10. The life of one packet. Acknowledgements double as the swarm's only evidence that it is still connected, which is what the lost-link failsafe listens to.

5.7 Traffic, acknowledgements and custody

Packets actually move (Fig. 10). Every airborne aircraft sends telemetry at 2 Hz, dropped once older than 2 s, and each surveyed site produces 12 chunks of imagery and situational data. Each hop is link-layer ARQ with up to four tries, so a hop that succeeds on attempt a costs

$$t_{\text{hop}} = \sum_{k=0}^{a-1} (4.1 + 2k) \text{ ms} + 1.5 \text{ ms}, \quad (15)$$

where 4.1 ms is the airtime of a 256-byte frame at about 500 kbit/s. A bad link therefore shows up as latency before it shows up as loss. Survey chunks are delay-tolerant [7]: with no route they wait in custody on the aircraft holding them, and an aircraft that lands hands its backlog to the GCS over the wire. Every delivered packet is acknowledged, and those acknowledgements are the only way an aircraft knows it still has a link home. Sixty seconds without one triggers the lost-link return.

5.8 Safety

Every setpoint is projected at least 40 m inside the geofence and under the AGL ceiling before the controller sees it. Separation assurance predicts each pair's closest point of approach over the next 4 s. With $\mathbf{r}$ and $\mathbf{v}$ the relative position and velocity (vertical axis weighted by 1.6),

$$t^* = \text{clip}\left(-\frac{\mathbf{r} \cdot \mathbf{v}}{\|\mathbf{v}\|^2}, 0, 4\right), \quad d = \|\mathbf{r} + \mathbf{v}t^*\|. \quad (16)$$

If d is under $R = 30$ m (8 m near the pads), the pair is pushed apart along the miss vector by $0.9R(R - d)/R$ and split vertically,

Table 3. How each hidden disturbance is noticed.

Event	Evidence	Conclusion
UAV failure	nobody hears it for 200 ms	failover; task freed after 5 s
Radio outage	same silence; own packets unacked	treated as failure; home after 60 s
Interference	noise-floor rise; DF bearings	source cross-fixed, fed to planning
Area RF drop, link cut	floor up everywhere; links fall	chain re-sized, re-routed
Motor or cell fault	current above thrust model	$\hat{k}$ rises: earlier RTH
Heavy rain	RSSI below map; camera loses ground	path-loss offset; cloud ceiling
GNSS drift	radio ranging disagrees	terrain-relative nav
New emergency	report reaches GCS	tasking, pre-emption

higher one up. The fence and terrain floor are then re-applied, so avoidance can never push an aircraft out of bounds. Take-offs are spaced 4 s apart. In all 24 benchmark runs there were no collisions, geofence violations, terrain strikes or flat batteries.

6 KNOWING WITHOUT BEING TOLD

A simulator makes cheating easy. The first version of this code, like many swarm demos, read the injected fault directly: “if this drone is marked dead, re-elect”. That makes a fault look handled while a real swarm would still be finding out about it. We removed every such path. A disturbance now changes physics or radio and nothing else; no flag is set and no callback fires. Table 3 lists what the swarm actually sees. A test file audits the autonomy’s source for any read of simulator state, and behavioural tests check that each disturbance is noticed rather than announced.

Estimators. Beliefs are simple and cheap. Wind is an exponential average of the aircraft’s own estimates (time constant 20 s), scaled to cruise height by the shear law. The path-loss offset is $\delta \leftarrow \delta + 0.3 (\text{med}(P_r^{\text{map}} - P_r^{\text{meas}}) - 1 - \delta)$ once a second over up to ten links, which is how rain is noticed. GNSS drift is caught when radio ranging to other aircraft and to the GCS disagrees with GNSS ranges by more than a threshold. Ranging to the GCS was added after we found that a drift common to every aircraft is invisible to ranging between aircraft.

Where is the interference? Each receiver reports its noise floor, and a rise over 6 dB counts as interference. Each aircraft’s direction-finding array also reports bearings to strong emitters, with noise that grows as the signal weakens and a fixed calibration bias. The swarm scores candidate cells $\mathbf{c}$ on a grid by how well one emitter explains the readings. For aircraft k in the active set $\mathcal{A}$ with measured interference J_k and predicted path loss $\ell_k(\mathbf{c})$ (free space plus terrain, from the map it carries),

$$\hat{P}(\mathbf{c}) = \frac{1}{|\mathcal{A}|} \sum_{k \in \mathcal{A}} (J_k + \ell_k(\mathbf{c})),$$

$$E(\mathbf{c}) = \sum_{k \in \mathcal{A}} (J_k + \ell_k(\mathbf{c}) - \hat{P}(\mathbf{c}))^2 + E_{\text{silent}} + E_{\text{DF}}, \quad (17)$$

where E_{silent} penalises cells that would have raised the floor at aircraft which heard *nothing*, and E_{DF} scores how well the cell agrees with the measured bearings. A ridge that shields one aircraft is evidence here, not noise. The best cell, its power and an uncertainty radius go to the relay optimiser, which then plans around it (Fig. 11).

Why is this link failing? Low delivery on an important link could be terrain, range, interference, a bad antenna angle or weather, and the right response differs: climb, move closer,

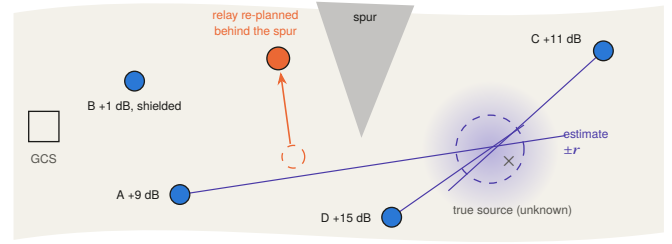


Fig. 11. Localising interference from noise rises and DF bearings (purple). The swarm never sees the true source (x). Its estimate, with a radius, enters the relay objective, which moves the relay so the spur sits between it and the source. B hears almost nothing, and that silence also counts as evidence.

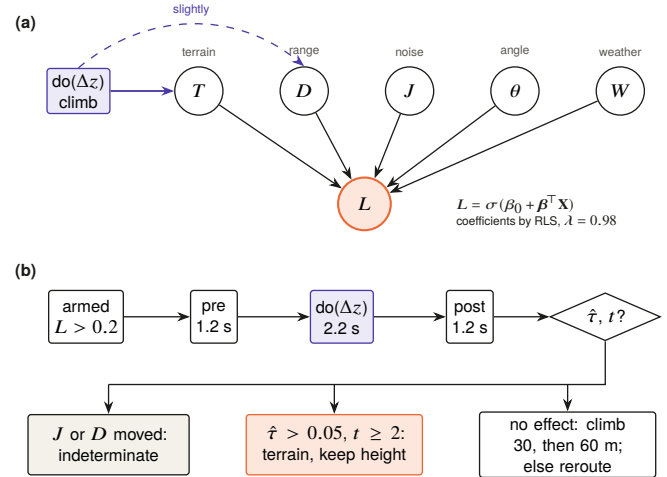


Fig. 12. (a) The structural causal model of link loss, with the altitude intervention acting on terrain occlusion. (b) One experiment. The swarm keeps a climb only when a clean, unconfounded measurement says terrain was the cause.

reroute, turn, or accept it. A structural causal model [6] relates normalised terrain occlusion T , distance D , noise rise J , antenna misalignment θ and weather W to loss L (Fig. 12a):

$$L = \sigma(\beta_0 + \beta_T T + \beta_D D + \beta_J J + \beta_\theta \theta + \beta_W W). \quad (18)$$

The coefficients are tracked online by recursive least squares with forgetting factor $\lambda = 0.98$ [13], using the sigmoid slope as a Gauss-Newton weight:

$$\mathbf{K} = \frac{\mathbf{P}\mathbf{x}}{\lambda + \mathbf{x}^\top \mathbf{P}\mathbf{x}}, \quad \beta \leftarrow \Pi_{\mathcal{B}}[\beta + \mathbf{K}(y - \hat{y})\hat{y}(1 - \hat{y})], \quad (19)$$

$$\mathbf{P} \leftarrow (\mathbf{P} - \mathbf{K}\mathbf{x}^\top \mathbf{P})/\lambda.$$

$\Pi_{\mathcal{B}}$ clips each coefficient to its physical sign (more occlusion, range or noise can only hurt). Without it the regressors, which are strongly collinear in normal flight, let RLS slide to values that fit equally well and mean nothing. The root cause is the parent with the largest $|\beta_k X_k|$.

Correlation is not enough to justify a climb, so the swarm runs a small experiment. It records the loss, commands $\text{do}(\Delta z)$ on the relay end of the link, and measures again (Fig. 12b). The effect $\hat{\tau} = \bar{L}_{\text{post}} - \bar{L}_{\text{pre}}$ counts only if it exceeds 0.05 with $t \geq 2$. If J moved by more than 2.5 dB or range by more than 8% during the window, the test is confounded and reported as indeterminate rather than guessed. A null result escalates the climb from 15 to 30 to 60 m before concluding “not terrain”, since a 15 m climb barely changes the geometry under a 600 m ridge. Only one experiment runs at a time; overlapping climbs contaminated each other’s measurements when we allowed more.

What we got from the experiments is honest but modest. The causal layer never blamed terrain for interference in our tests. It

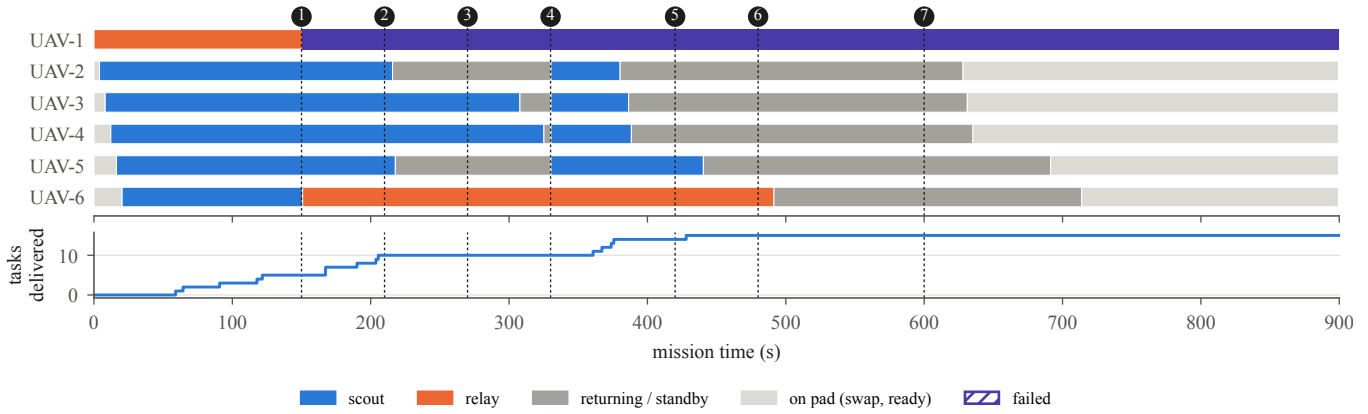


Fig. 13. One Kedarnath run, from its logs. Disturbances: **1** relay UAV-1 fails, **2** GCS receiver outage, **3** 30% packet loss, **4** P1 region reported, **5** interference zone, **6** relay battery fault, **7** late P1 report. At 1 the swarm notices UAV-1's silence and promotes UAV-6 into its slot. At 4 two scouts heading home are turned round, and the region is delivered by about 430 s. After 6 the faulty relay's measured drain rises and it comes home early. Event 7 arrives with 300 s left and its round trip does not fit, so the planner declines it.

confirms genuine terrain shadow only about 20% of the time, because climbing one end of a link lifts the ray over a far ridge by very little. The routing and interference layers do not depend on it being right; they use it as a stability hint.

Two more things surprised us. A relay whose radio fails and one that crashes look identical from outside, so the swarm handles both the same way; when the radio comes back the aircraft is heard, rejoins, and the role manager notices it has one relay too many. And honesty cost a little. When the energy model switched from the true wind to the swarm's own estimate, completion fell from 98.4% to 93.5%. The calibration had been fitted to ground-level wind, while the swarm now measures the stronger wind at cruise height. Refitting airspeed on the same logged flights brought it back to 97.3%.

7 EVALUATION

7.1 Scenarios, baseline and metrics

A scenario is a JSON file fixing the terrain, seed, fleet, GCS, geofence, tasks, clock and a timeline of hidden disturbances. Positions can be given in metres or as a fraction along the valley, so one scenario runs on any site. The disturbance types cover everything listed for Stage 2. Table 4 summarises the four we ship with their results.

The **baseline** keeps everything except the adaptive role layer: same flight control, energy-aware return, geofence, placement network and packet model. Roles are fixed at launch (one relay, the rest scouts) and aircraft relaunch in their original role. A failed relay is not replaced, a new P1 task waits for a free scout, nothing hands over and nobody stands by. The gap between the two is what the adaptive autonomy is worth. Each scenario runs with three seeds, which vary fading, packet outcomes and, in the stress case, task placement. Metrics follow the categories in the problem statement and are computed from simulator truth, never from what the swarm believes. The priority-weighted score is

$$S = \sum_p w_p \delta_p / \sum_p w_p, \quad (20)$$

with $\delta_p = 1$ if task p 's data reached the GCS by the deadline. Availability is airborne time with an end-to-end path of reliability at least 0.5; recovery time runs from a disruption until every airborne aircraft has had a path for 1 s; reconfiguration efficiency is one minus the share of role changes undone within 30 s.

7.2 Results

Fig. 14 compares the two strategies over 12 runs each. The adaptive swarm is ahead on every mission and communication metric: completion 97.3% against 87.8%, priority-weighted score 96.7% against 84.0%, PDR 96.7% against 91.6% and availability 95.3% against 88.6%. The biggest gap is how quickly a new emergency is answered, 42 s against 180 s, because the baseline waits for a scout to become free while ours pre-emptively one or turns one round. Downtime (124 s against 273 s) and recovery time (6.7 s against 13.4 s) are both roughly halved, because the chain is re-sized and re-staffed instead of being left as launched. After the first fault, availability stays at 93.9% against 84.5%.

Fig. 13 follows one Kedarnath run, and Table 4 breaks the numbers down. Uttarkashi is a draw with the baseline: the Bhagirathi valley there is straight and open, one relay covers everything, and adaptation has little to win, which is what it should look like. The stress scenario is hardest on communication; its 13 s mean recovery is dominated by a 60 s severed link and a 25 s GCS outage that no re-planning can fix, only wait out. The adaptive swarm also flies its batteries harder, down to 10.5% in Kedarnath against a baseline average of 34%. That is still above the 8% landing reserve, and it is the price of keeping aircraft working instead of parking them.

7.3 What did not work

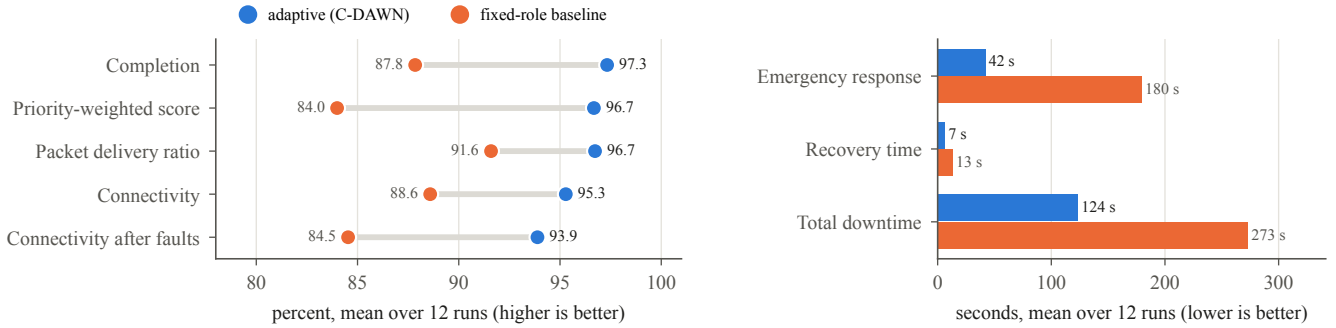
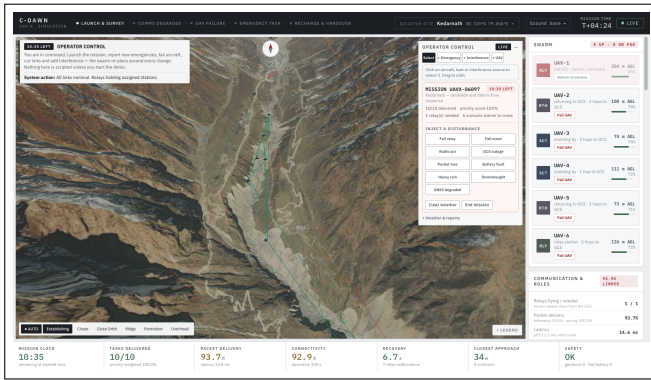
Our first energy model was about 1.8 times too pessimistic and sent full-battery scouts home before they had done anything. Tasks were once left assigned to a scout that had died, so nobody else picked them up; they are now released after 5 s of silence. A handover once never completed because the placement network kept moving the replacement away from the station it was meant to take, which is why a relay being relieved is now fixed in the plan. The liquid time-constant controller [5] we first proposed for flight control lost to a well-tuned cascaded PID in 16 of 16 paired gust trials (0.62 m against 0.18 m cross-track error), so the PID flies and the LTC runs in shadow.

8 SOFTWARE AND REPRODUCIBILITY

The code is Python 3.10+ with PyTorch for the placement network, FastAPI and WebSockets for the GCS server, and React with Three.js for the operator console (Fig. 15). Nothing needs a GPU or an internet connection once installed: terrain packs and trained weights ship in the repository. A live mission runs with `python run_node.py --scenario kedarnath_landslide --autostart`; a headless run at

Table 4. Scenarios and adaptive results per scenario (mean of three seeds). Tasks are those at launch plus those released later.

Scenario	UAVs	Tasks	Clock	Hidden events	Completion	PDR	Availability	Recovery	Min. battery
Quickstart, synthetic valley	5	8 + 1	12 min	4	100.0%	95.4%	93.9%	4.6 s	24.1%
Kedarnath landslide (real terrain)	6	10 + 6	15 min	8	93.8%	97.2%	95.8%	8.9 s	10.5%
Uttarkashi earthquake (real terrain)	5	9 + 6	15 min	7	100.0%	100.0%	100.0%	0.0 s	29.4%
Stress, synthetic valley	5	8 + 7	15 min	11	95.6%	94.3%	91.5%	13.3 s	21.3%

**Fig. 14.** Adaptive C-DAWN against the fixed-role baseline, mean of 12 runs each (4 scenarios $\times$ 3 seeds). Left: percentages (dots, since the axis is zoomed). Right: times in seconds. Neither strategy had a collision, a geofence violation or a flat battery.**Fig. 15.** The operator console during a live Kedarnath run: real terrain, geofence, relay chain, survey sites by priority, the swarm list with hop counts, and the challenge metrics along the bottom. Disturbances can be injected live from the same console.

5 to 9 times real time with `python run_scenario.py scenarios/<name>.json`; and the full benchmark in Table 4 with `python -m bench.uavx_suite`.

The same code can be split across three laptops: one owns the aircraft and physics, one runs relay placement and causal diagnosis on the live stream, and one writes situation reports. Work falls back to the first if another goes quiet for 1.5 s. Every run writes a standard log set (metadata with the code version, the resolved scenario, an event stream, 1 Hz aircraft and link tables, a per-packet log and the metrics). When the organisers publish their log format, `mission/recorder.py` is the single adapter point. 158 automated tests cover physics, radio, GNN equivariance, failover, handover, traffic, every disturbance type and the truth-boundary audit.

9 LIMITATIONS AND STAGE 2 PLAN

C-DAWN runs on its own simulator, which the problem statement allows in place of PX4, Gazebo or ns-3; it lets terrain, radio and flight share one model. The autonomy only ever talks to an aircraft through setpoints, so for Stage 2 we plan a PX4 SITL and ROS 2 bridge that replaces the physics and controller hooks and leaves the autonomy untouched. Also on the list: a contention-aware MAC, since airtime is modelled per hop but not shared; the organisers' log format; position from a proper es-

imator rather than truth plus an injected GNSS offset; and air density that changes with altitude. Survey detections currently come from each site's category rather than imagery, and the situation reports are templates.

10 CONCLUSION

A swarm that surveys a disaster valley is only useful if what it sees gets home. By treating the link to the GCS as part of the mission, sizing the relay chain from the terrain, placing relays with a network trained through the physics itself, handing stations over before a battery forces the issue, and making the swarm find out about trouble the way a fielded one would, C-DAWN delivered 97% of its tasks through 30 hidden disturbances without losing an aircraft to its own planning. The next step is the same autonomy on PX4, and after that on real airframes.

REFERENCES

- [1] ITU-R, "Propagation by diffraction," Rec. P.526-15, 2019.
- [2] H. T. Friis, "A note on a simple transmission formula," *Proc. IRE*, vol. 34, no. 5, pp. 254 to 256, 1946.
- [3] V. G. Satorras, E. Hoogeboom and M. Welling, "E(n) equivariant graph neural networks," in *Proc. ICML*, 2021.
- [4] I. Bekmezci, O. K. Sahingoz and S. Temel, "Flying ad-hoc networks (FANETs): A survey," *Ad Hoc Networks*, vol. 11, no. 3, pp. 1254 to 1270, 2013.
- [5] R. Hasani, M. Lechner, A. Amini, D. Rus and R. Grosu, "Liquid time-constant networks," in *Proc. AAAI*, 2021.
- [6] J. Pearl, *Causality: Models, Reasoning and Inference*, 2nd ed. Cambridge Univ. Press, 2009.
- [7] K. Fall, "A delay-tolerant network architecture for challenged internets," in *Proc. ACM SIGCOMM*, 2003.
- [8] U.S. Department of Defense, "Flying qualities of piloted aircraft," MIL-F-8785C, 1980.
- [9] European Space Agency, "Copernicus DEM," and AWS Open Data Terrain Tiles, accessed 2026.
- [10] EOX IT Services, "Sentinel-2 cloudless," s2maps.eu, CC BY 4.0.
- [11] J. Scherer *et al.*, "An autonomous multi-UAV system for search and rescue," in *Proc. ACM DroNet*, 2015.
- [12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [13] S. Haykin, *Adaptive Filter Theory*, 5th ed. Pearson, 2014.
- [14] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269 to 271, 1959.